

CNN Debugging

1. Overview

This report evaluates InitialCNN, a convolutional neural network comprising four convolutional layers (sigmoid activation, max-pooling) followed by two fully connected layers with no explicit softmax output layer, trained on the CIFAR-10 dataset. CIFAR-10 consists of 60,000 colour images (32×32 pixels) across 10 classes: 50,000 for training and 10,000 for testing (*sample images shown in Figure A1 in Appendix*). The dataset comes perfectly balanced, with each class contributing exactly 10% of both the training and test sets. The train/test split is pre-defined by the dataset creators and loaded via separate flags, preventing any possibility of data leakage. A fixed random seed of 42 is set across Python, NumPy and PyTorch to ensure reproducibility. Normalisation is applied using fixed values of mean=(0.5, 0.5, 0.5) and std=(0.5, 0.5, 0.5) for all channels (a simplification approach). The network is trained for 15 epochs using the Adam optimiser at a learning rate of 0.001, with cross-entropy loss. The evaluation reports training loss, training accuracy, test loss and test accuracy across all 15 epochs, and analyses learning behaviour through learning curves and per-layer gradient magnitude plots. Final performance is additionally assessed using F1 score and AUC-ROC on the test set.

A central finding of this analysis is that the results can only be correctly interpreted by considering the interaction between three components: the sigmoid activation function, which weakens gradient signals in early layers; the Adam optimiser, which compensates by adapting each weight's learning rate individually; and the volume of training updates - 390 batches per epoch over 15 epochs, totalling 5,850 weight updates - which allows even small, weakened gradients to accumulate into meaningful learning over time.

2. Evaluation Metrics

Complete epoch-by-epoch results are shown in Table A1 in the appendix.

Starting at 17.66% training accuracy in epoch 1, close to the 10% random baseline for a 10-class problem, the model improved steadily to training accuracy of 67.43% by epoch 15. Test accuracy increased from 26.68% to 63.17%. The Train Loss fell from 2.1637 to 0.9161, and the Test Loss fell from 1.9575 to 1.0381. The macro F1 score of 0.629 is consistent with the 63.17% test accuracy, as expected for a balanced dataset. Per-class F1 reveals where performance is concentrated: strongest on automobile (0.745), ship (0.748) and frog (0.709), weakest on cat (0.472) and dog (0.499). The confusion matrix confirms that cats and dogs are heavily confused with each other (*see Figure A2 in Appendix for confusion matrix*). AUC-ROC of 0.937 indicates the model ranks the correct class highly even when it misclassifies.

3. Learning Curves

As can be seen in Figure 1(A), Both loss curves drop steeply between epochs 1 and 5, then transition into a slower decline. Test loss begins below train loss then cross around epoch 9. Both accuracy curves rise steeply early then slow. Test accuracy begins above train accuracy then cross around epoch 9. The same pattern appears in both plots: test starts ahead of train, the lines cross around epoch 9, and then train pulls ahead. Test accuracy starts higher and loss starts lower in early epochs because training accuracy is averaged over the entire epoch, including the weak early batches when the model has barely learned anything, while test accuracy and loss are measured after all updates are complete, reflecting the model at its strongest point in that epoch. After epoch 9, the model begins fitting training-specific details that do not generalise to unseen images and that's when train pulls ahead. This is overfitting, and the widening gap between the two lines is its signature. Critically, as can be seen in Figure 1, neither curve had flattened by epoch 15, confirming the network had not yet converged and would continue to improve with further training.

4. Gradient Magnitude Analysis

See Figure A3 in appendix for more detailed gradient figures.

While the learning curves show that the network learned, the gradient magnitude plots in figure 1(B) reveal how strongly each layer was being updated at each point in training. As outlined in the Overview, three components interact to shape these results: sigmoid causing vanishing gradients, Adam compensating per update, and training volume allowing accumulation over time.

Most layers show gradients continually rising from near zero to higher values by epoch 15. Rising gradients reflect increasingly coherent error signals as learning progresses (not divergence from the optimum), which is

confirmed by the simultaneously falling loss curve. The fact that all layers except FC2 are still rising at epoch 15 confirms the network had not converged, consistent with the accuracy curves showing no plateau by the final epoch. Three layers tell the story most clearly:

FC2.W is the only layer whose gradient decreases, falling from ~ 0.72 to ~ 0.52 by epoch 15. As the layer closest to the loss, it receives the strongest, most direct error signal from the start, undiluted by sigmoid layers. It learns first, and as its weights settle, less correction is needed and the signal naturally shrinks.

Conv2.W shows the largest absolute growth, rising from near zero (~ 0.04) to ~ 0.75 , reflecting Adam progressively amplifying a weak signal until it becomes one of the strongest in the network.

Conv1.W stays closest to zero throughout, ending at only ~ 0.24 . The gap between conv1.W (0.0178) and fc2.W (~ 0.72) at epoch 1, approximately 40 times, is the vanishing gradient problem made visible. Adam increases conv1's effective step size to compensate but cannot restore the precision of the direction information degraded by five sigmoid layers. Conv1 learns, but more slowly and less reliably than layers closer to the output.

The vanishing gradient problem is real and measurable. Adam prevented it from being catastrophic by allowing every layer to receive meaningful updates and all gradients grew over time, but the unevenness remained throughout training.

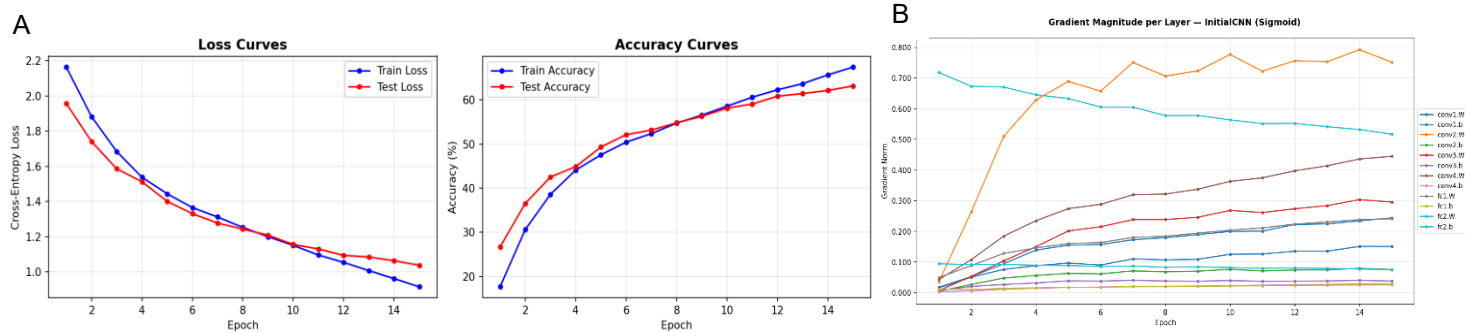


Figure 1. (A) The learning curves plotting loss and accuracy for each epoch across 15 epoch training run for InitialCNN. Both metrics are tracked separately for training set (blue) and test set (red). (B) Average gradient norm per parameter across epochs 1–15 for InitialCNN.

5. Discussion

InitialCNN demonstrates that architectural weaknesses and optimiser strengths interact in ways that are not visible from accuracy numbers alone. The gradient magnitude plots are essential for understanding what is happening inside the network during training. The 63% result is best understood as the product of three interacting forces: sigmoid creating a problem, Adam partially compensating for it per update, and 5,850 total weight updates giving the early layers enough repetitions to learn despite the weakened signal.

6. Architecture Improvement

The two problems highlighted in InitialCNN were vanishing gradients caused by sigmoid activations, and a growing train/test gap signalling overfitting. Two targeted changes can address these directly. Sigmoid can be replaced with ReLU in every hidden layer. ReLU's derivative is 1 for positive inputs, eliminating gradient shrinkage at the source rather than compensating for it through more epochs or a higher learning rate. Additionally, one Dropout layer can be added after FC1, randomly disabling 50% of neurons per batch during training, preventing the network from memorising training-specific patterns and overfitting. Finally, normalisation is corrected and mean and standard deviation are computed from the training set only and applied to both train and test sets, replacing the simplified fixed values used in InitialCNN. It should be noted that in vanilla CNN architecture, a softmax output layer is considered a standard component. Also note that adding an explicit softmax layer while using PyTorch's CrossEntropyLoss would apply softmax twice, since PyTorch's CrossEntropyLoss already applies log-softmax internally (*see figure A4 in appendix for PyTorch documentation*).

7. Evaluation Metrics and Comparison

Complete epoch-by-epoch results are shown in Tables A2 in the Appendix.

Starting at 39.55% training accuracy and 53.45% test accuracy in epoch 1, well above the 10% random baseline and InitialCNN's accuracy at the same point, the new model (hereinto: “new model” or “ImprovedCNN”) improved rapidly to 95.01% training accuracy and

75.20% test accuracy by epoch 15. This is a 12.03 percentage point gain in test accuracy over InitialCNN. The new model train loss fell from 1.6225 to 0.1426, a final value 84.4% lower than InitialCNN's final value, reflecting

far more effective gradient flow throughout training. Final test loss is marginally higher in the new model, rising from 1.0381 in InitialCNN to 1.0925, despite better accuracy. This may be a consequence of overfitting after epoch ~6. The macro F1 of 0.753 and AUC-ROC of 0.968 are consistent with the test accuracy and confirm the improvement across all classes, up from 0.629 and 0.937 respectively. Per-class F1 shows the same relative ordering as InitialCNN: strongest on automobile (0.865), ship (0.848) and frog switched to truck (0.840), weakest on cat (0.567) and bird (0.657), confirming that visually similar classes remain harder regardless of architecture. The one area of concern is the train/test gap, which widened from 4.26% to 19.81%, indicating the new model learns faster and reaches a higher ceiling, but also overfits more aggressively within the same 15 epochs. Table 1 summarises the final metrics for both models.

Metric	InitialCNN	ImprovedCNN	Change	
Final Train Loss	0.9161	0.1426	-0.7735	↓
Final Test Loss	1.0381	1.0925	+0.0544	⚠
Final Train Accuracy	67.43%	95.01%	+27.58%	↑
Final Test Accuracy	63.17%	75.20%	+12.03%	↑
Train/Test Gap	4.26%	19.81%	+15.55%	⚠
Macro F1	0.629	0.753	+0.124	↑
Macro AUC-ROC	0.937	0.968	+0.031	↑

Table 1: InitialCNN vs ImprovedCNN final metrics. Green = improvement, yellow = area of concern.

8. Learning Curves

Speed and accuracy of learning: As visualized in Figure 2(A), ImprovedCNN reaches peak test accuracy of 77% at epoch 10, a level InitialCNN never achieved at all, while InitialCNN was still steadily improving at epoch 15 with no sign of plateauing. This indicates that the new model learns both faster and to a higher ceiling. The loss curves of the new model drop far more steeply than those of the initial model, reflecting the immediate full-strength gradient signals ReLU provides.

Overfitting patterns: Both models show the same crossover pattern in which the test starts ahead of train in early epochs, the lines cross, and then train pulls ahead. In InitialCNN the gap that opens after the crossover is small because learning is slow overall. In the new model the divergence begins earlier, and the gap widens more aggressively. By epoch 5 the network begins memorising training-specific details (overfitting) possibly alongside genuine learning. Test accuracy continues rising until epoch 10 then begins drifting lower, suggesting the model has exhausted generalisable patterns and memorisation dominates which produces the widening train/test accuracy gap.

Convergence: InitialCNN has not fully converged by epoch 15 as both loss and accuracy are still rising, albeit gradually, consistent with early layers still accumulating small updates and the model not yet reaching its ceiling. In contrast, in the new model, test accuracy peaks around epoch 10 and begins drifting lower, while test loss has been rising since epoch 9. This signals that the model has passed its optimal point, it converged on the training data and then continued into overfitting. Further training beyond epoch 10 is actively harmful rather than neutral, making early stopping the appropriate intervention.

9. Gradient Magnitude Analysis

See Figure A3 in appendix for more detailed gradient figures.

Three layers are highlighted in Figure 2(B) for direct comparison between the effect of sigmoid and ReLU on gradients. In InitialCNN, conv1.W started at ~0.02 and grew slowly to ~0.24, starved by five sigmoid layers. In the new model, conv1.W started at ~1.14, approximately 57 times larger from epoch 1, and stabilised around 1.92. The vanishing gradient problem is resolved, and the first layer receives a signal as strong as the last. Conv2.W shows the largest absolute growth in InitialCNN (~0.04 to ~0.75), reflecting Adam’s progressive amplification of a weak signal. In the new model, conv2.W starts higher (~1.45) and remains strong throughout, with no need for progressive amplification. Notably, by epoch 3 conv2.W in the new model begins declining, mirroring the behaviour of FC2 in both models. This is a sign that the layer has found a good configuration and requires less correction over time. FC2.W decreases over time in both models as the output layer converges first. In the new model, this decrease is steeper and begins earlier, reflecting the network’s rapid learning.

Overall, in InitialCNN, gradients span a 40× range from conv1 to fc2 at epoch 1 and all layers are still rising at epoch 15. In the new model, all layers start at comparable magnitudes and most stabilise by epoch 10. The gradient plots are the most direct evidence that ReLU solved the problem sigmoid created.

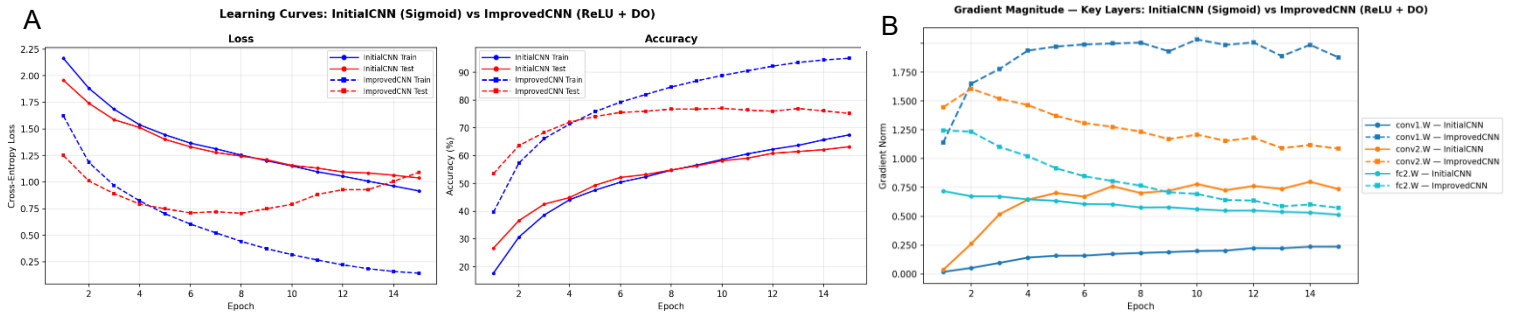


Figure 2. (A) The learning curves plotting loss and accuracy for each epoch across 15 epoch training run for InitialCNN (solid) and ImprovedCNN (dashed). Both metrics are tracked separately for training set (blue) and test set (red). (B) Average gradient norm for key parameter across epochs 1–15 for InitialCNN (solid) and ImprovedCNN (dashed).

10. Discussion

Replacing sigmoid with ReLU resolved vanishing gradients and enabled full-strength learning from epoch 1, raising test accuracy significantly. Adding Dropout improved early generalisation but could not counteract the overfitting driven by the network's faster and more powerful learning. Reducing the number of training epochs to around 10 might preserve the best test performance.

Comparison to MLP

InitialMLP is a fully connected network comprising two hidden layers (512 and 256 neurons, both ReLU) followed by a 10-class output, trained on the same CIFAR-10 dataset with identical settings: Adam optimiser, learning rate 0.001, batch size 128, 15 epochs. Each image is flattened from $32 \times 32 \times 3$ into a 3,072-element vector before being passed through the network. The report includes evaluation and comparison of accuracy, parameter count, hyperparameter tuning, training speed, memory usage, and weight interpretability to the new CNN model improved from the InitialCNN.

1. Evaluation Metrics and Comparison

Complete epoch-by-epoch results are shown in Tables A3 in the Appendix.

Starting at 42.20% training accuracy and 48.15% test accuracy in epoch 1, the MLP reached 84.42% training accuracy and 53.18% test accuracy by epoch 15, a 22.02 percentage point gap behind the CNN's 75.20%. Train loss fell to 0.4407 versus the CNN's 0.1426, and test loss fell initially to 1.34 by epoch 4 then rose continuously to 1.98 (the CNN's final test loss was 1.09). The MLP's train/test gap of 31.24% versus the CNN's 19.81% confirms heavier memorisation. Macro F1 of 0.537 and AUC-ROC of 0.890 are lower than the CNN's 0.753 and 0.968, consistent with the MLP lower accuracy. The confusion matrix (Figure 3(A)) shows the CNN's errors are concentrated in feature-similar pairs (cats with dogs, trucks with automobiles), while the MLP's errors are more diffuse, reflecting pixel-level statistical learning rather than feature-based understanding. The MLP's poor generalisation is structural due to the flattening which destroys all spatial relationships, leaving no mechanism to detect edges, textures or shapes.

Table 2 summarises the final metrics for the MLP and compares to the new CNN model.

2. Learning Curves

As shown in Figure 3(B), the CNN learns faster, for longer and reaches a higher ceiling. The MLP plateaus early and barely improves for the remaining ten epochs despite train accuracy continuing to climb, which signals that it is now memorising. Eventually the CNN also moves to overfitting but to a lesser degree. The CNN's train loss drops steeply while the MLP's falls more gradually, reflecting that the CNN extracts far more from each epoch. Both models follow the same test loss pattern dropping to a minimum around epoch 5 - 8 then rising as overfitting sets in. The difference is in scale and steepness as the CNN's minimum is lower, and while it rises gradually after that point, the MLP climbs much more steeply.

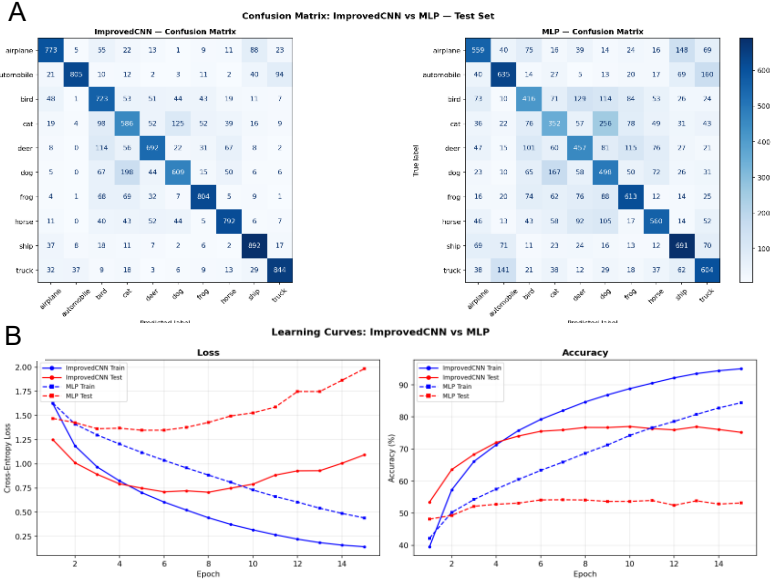


Figure 3: (A) Confusion matrices for ImprovedCNN and MLP. (B) Learning curves plotting loss and accuracy for each epoch across a 15-epoch training run for new CNN (solid) and MLP (dashed). Both metrics are tracked separately for training set (blue) and test set (red).

Metric	New CNN	MLP	Observation
Parameters	653,386	1,707,274	MLP has 2.6× more
Final Train Accuracy	95.01%	84.42%	CNN +10.59%
Final Test Accuracy	75.20%	53.18%	CNN +22.02%
Final Train Loss	0.1426	0.4407	CNN 69% lower
Final Test Loss	1.0925	1.9829	CNN 45% lower
Train/Test Gap	19.81%	31.24%	MLP overfits more
Macro F1	0.753	0.537	CNN +0.216
Macro AUC-ROC	0.968	0.890	CNN +0.078
Epoch time (GPU)	12.7s	12.2s	Comparable on GPU
Peak GPU memory	133.9 MB	73.6 MB	CNN uses 1.8× more

Table 2: New CNN vs InitialMLP final metrics. Green = better. Yellow = area of concern.

3. Parameters and Hyperparameter Tuning

The MLP has 2.6 times more parameters than the CNN yet performs worse. CNNs achieve this efficiency through weight sharing whereby a single $3 \times 3 \times 3$ conv1 filter (27 weights) scans the entire image, detecting the same feature at every position. The MLP needs separate weights for every pixel-to-neuron connection, increasing its parameter count and memorizing pixel level features. This high parameter counts also contributes to the high overfitting.

Both models used identical hyperparameters, to isolate architecture as the variable. The results suggest each model would benefit from different tuning directions. The MLP's 31.24% train/test gap points to a need for stronger regularisation (lower learning rate, higher dropout, or weight decay). The CNN's test accuracy plateaued around epoch 10 before harmful overfitting emerged suggesting that early stopping at that point, or a learning rate schedule that decays after epoch 10 would preserve its best generalisation.

4. Training Speed and Memory

On GPU, training speed is comparable between the models. GPU parallelisation eliminates the advantage matrix operations have over convolutions, unlike on CPU where the CNN is approximately $2 \times$ slower. Memory usage differs as the CNN uses $1.8 \times$ more memory, as it stores intermediate feature maps from all conv layers during the forward pass for use later in backpropagation, while the MLP stores only flat vectors. Speed and memory are the two areas where the MLP holds a practical advantage.

5. Weight Interpretability

The CNN's conv1 filters (Figure 4(A)) are 3×3 patches across three colour channels which emerged entirely from backpropagation. Each filter corresponds to a concrete visual operation of scanning a small spatial window for a specific colour pattern thereby making them interpretable as colour and contrast detectors. Colour is the dominant discriminative feature in CIFAR-10, which is why the filters reflect colour structure rather than the oriented edges seen in higher-resolution datasets.

The MLP's fc1 weights (Figure 4(B)), when reshaped back and displayed as images, show pure noise. This is inherent to how the MLP learns. Each neuron connects to all 3,072 pixels simultaneously, so the network cannot concentrate learning into spatially coherent patterns. The

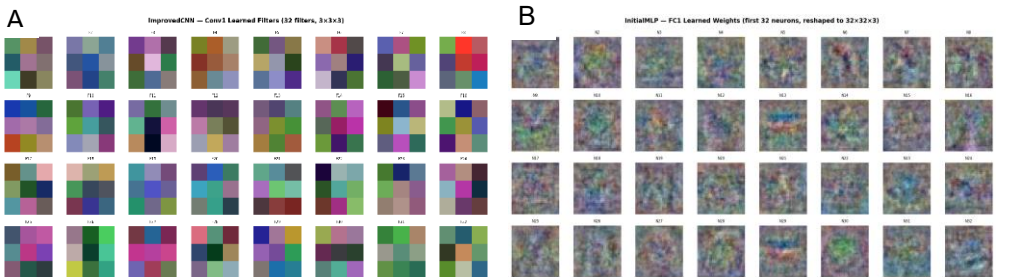


Figure 4: (A) Learned conv1 filters for ImprovedCNN (64 filters, $3 \times 3 \times 3$). (B) FCI learned weights for InitialMLP (first 32 neurons, reshaped to $32 \times 32 \times 3$).

result is statistical correlations between pixel values and class labels rather than visual features, which are uninterpretable and generalise poorly.

6. Which Architecture Suits Which Tasks

The filters above reflect a deeper architectural difference in what each network is built for. CNNs are suited to data with spatial structure (images, video, etc.). A convolutional filter detects the same feature regardless of where it appears in the image. This translation invariance directly matches how visual information is structured. CNNs can also scale efficiently to any image size with the same parameter count. MLPs are suited to tabular or structured data where input position carries no spatial meaning. In these settings there is no spatial structure to exploit, no benefit to weight sharing, and an MLP's ability to learn arbitrary non-linear mappings is useful.

7. Discussion

The MLP's flattening destroys spatial relationships before the network processes the data, leaving no mechanism for local feature detection. This produces noise-like weights, heavy memorisation (31.24% train/test gap), and 22 percentage points lower test accuracy than the CNN on image classification. The same structural simplicity gives the MLP lower memory usage and comparable GPU speed which give it an advantage if used on the correct type of task.

Overfitting

1. Introduction

The new CNN demonstrates clear and measurable overfitting within the 15 training epochs. Using the learning curves in Figure 2(A), two transition points can be identified: epoch 5 where the model moves from underfitting to overfitting as train overtakes test and the gap starts widening, and epoch 10 where overfitting becomes actively harmful to accuracy.

Between epochs 1 and 5 the model is in a good generalisation state where both train and test accuracy are rising, loss is falling, and the network is learning genuinely useful patterns. At epoch 5 as overfitting begins test accuracy continues rising as genuine learning and memorisation occur simultaneously, though the train/test loss gap starts widening as train loss continues falling steeply while test loss flattens, signalling the model is increasingly fitting training-specific details that don't generalise and may be less reliable when deployed on new data. Test accuracy eventually peaks at 77.03% at epoch 10, then drifts lower to 75.20% by epoch 15, while train accuracy climbs to 95.01% and the train/test gap widens to 19.81%. Further training beyond epoch 10 is actively harmful to total accuracy.

A large gap signals that model capacity was spent memorising rather than learning generalisable features therefore the model's ceiling is artificially limited and its performance less reliable in deployment. The ideal is high accuracy with a small gap which makes evident that accuracy comes from genuine learning rather than memorisation. (InitialCNN's small gap of 4.26% should not be misread as better generalisation: it reflects incomplete learning, not robustness. ImprovedCNN's larger gap is the cost of faster, more powerful learning and reducing it is the goal of the two solutions below)

2. Solutions

Two solutions are evaluated to try to mitigate the overfitting in the new CNN model: early stopping and weight decay. Both are applied to the new CNN under identical conditions and compared against the baseline overfitted model. Results are shown in Table 3 and Figure 5.

Early Stopping

Early stopping monitors test loss of each epoch and stops training when it stops improving, as test loss captures increasing overconfidence before it shows up as accuracy drops. Loss and accuracy are the appropriate metrics for overfitting monitoring as they track generalisation directly across epochs, whereas metrics like F1 and AUC-ROC evaluate error type distributions and are better suited to final model evaluation than to detecting training deterioration. Note that ideally a separate validation set would be used as the stopping criterion, keeping the test set unseen until final evaluation. As the starter code which was the base for the new CNN provided only train and test splits, test loss was monitored directly, a simplification that introduces minor data leakage through the stopping decision. With patience=3, training continued until epoch 11 but the best model, saved at epoch 8, is what gets deployed: 77.04% test accuracy and a 6.75% gap, roughly half the baseline. Patience is itself a

hyperparameter requiring tuning as too low risks stopping during a temporary dip, too high allows unnecessary overfitting. The selected value was chosen after comparison.

Weight Decay

Weight decay penalises large weights throughout training. When neurons over-specialise to specific training examples their weights grow disproportionately large, and this penalty continuously pushes back against that, forcing more generalisable representations. With $\lambda=1e-3$, final test accuracy was 76.77% and the train/test gap reduced to 11.71%. By the later epochs, test loss stayed lower than the baseline, visible evidence that weight decay constrained memorisation from the start. λ is also a hyperparameter requiring tuning, as too small has negligible effect, too large prevents meaningful learning. The selected value was chosen after comparison.

Metric	Baseline	Early Stopping	Weight Decay	Best
Final Test Accuracy	75.20%	77.04%	76.77%	Early Stop
Final Train Accuracy	95.01%	83.79%	88.48%	—
Train/Test Gap	19.81%	6.75%	11.71%	Early Stop
Epochs trained	15	11 (best: 8)	15	—
Hyperparameter	—	patience = 3	$\lambda = 1e-3$	—

Table 3: Overfitting solutions comparison. Green = improvement over baseline. Yellow = area of concern.

3. Discussion

Both solutions meaningfully reduce the train/test gap while maintaining test accuracy above the baseline. Early stopping is reactive. It prevents the model from training past its optimal point but does nothing about overfitting that already occurred. Weight decay is proactive. It modifies how the model learns at every update throughout training, reducing the tendency to memorise from the start. When addressing overfitting, test accuracy alone is not sufficient to evaluate a model's quality. A model with 77% accuracy and a 9% gap is more trustworthy than one with 77% accuracy and a 20% gap, because the smaller gap indicates the accuracy comes from generalisable features rather than memorised training details. In deployment, where data differs slightly from the training distribution, the model with the smaller gap will degrade less.

Data Augmentation

1. Overview

Data augmentation reduces overfitting by exposing the model to more varied versions of the same training images. Each image is randomly transformed every epoch, so the model cannot memorize fixed pixel patterns, as if trained on a much larger dataset. Five augmentations were sampled. (See Figure A5 in Appendix for augmented images.) Three augmentations were then applied as a sequential pipeline exclusively to the training set: random crop (padding=4), random zoom/resize (scale 0.80–1.0), and horizontal flip (p=0.5). All three transforms are applied to every image in order each epoch, with each randomly sampled so the model sees a different version of each image every epoch. The test set received normalization only. Training was extended to 25 epochs to allow sufficient convergence under the harder, continuously varying training conditions.

2. Evaluation Metrics and Comparison

Complete epoch-by-epoch results are shown in Table A5 in the Appendix.

AugmentedCNN reached 81.85% final train accuracy and 80.80% final test accuracy by epoch 25, a 5.60 percentage point gain in test accuracy over the baseline's 75.20%, despite lower train accuracy (81.85% vs 95.01%). The train/test gap collapsed from 19.81% to 1.05%, essentially zero, the defining result of augmentation. Final test loss fell from 1.0925 to 0.5953, while train loss rose from 0.1426 to 0.5319, reflecting a harder training task that no longer permits memorization.

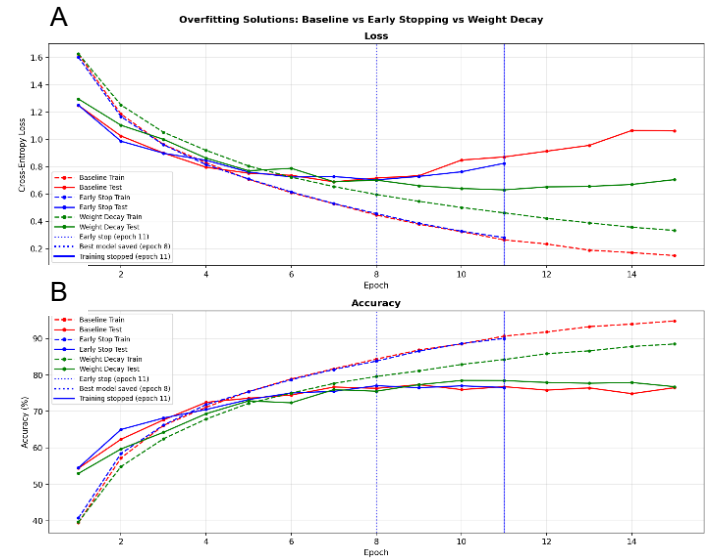


Figure 5: (A) Loss curves and (B) accuracy curves across 15 epochs for the baseline ImprovedCNN (red), early stopping (blue), and weight decay (green). Both metrics are tracked separately for training set (dashed) and test set (solid). Vertical markers indicate best model saved (epoch 8, dotted) and training stopped (epoch 11, solid).

Macro F1 improving from 0.753 to 0.805 reflects broad gains across all classes, confirmed by the per-class breakdown in Figure 6(A) where every class surpasses its baseline, not an average masking uneven improvement. AUC-ROC improving from 0.968 to 0.979 confirms the model's confidence rankings are better. The confusion matrices (Figure 6(B)) show that cat/dog confusion persists, suggesting these classes remain structurally ambiguous at 32×32 resolution, a limitation augmentation cannot resolve. Deer's recall improvement (+41.6pp), confirms that augmentation specifically addressed spatial sensitivity. The model learned position-invariant features rather than memorizing fixed pixel patterns.

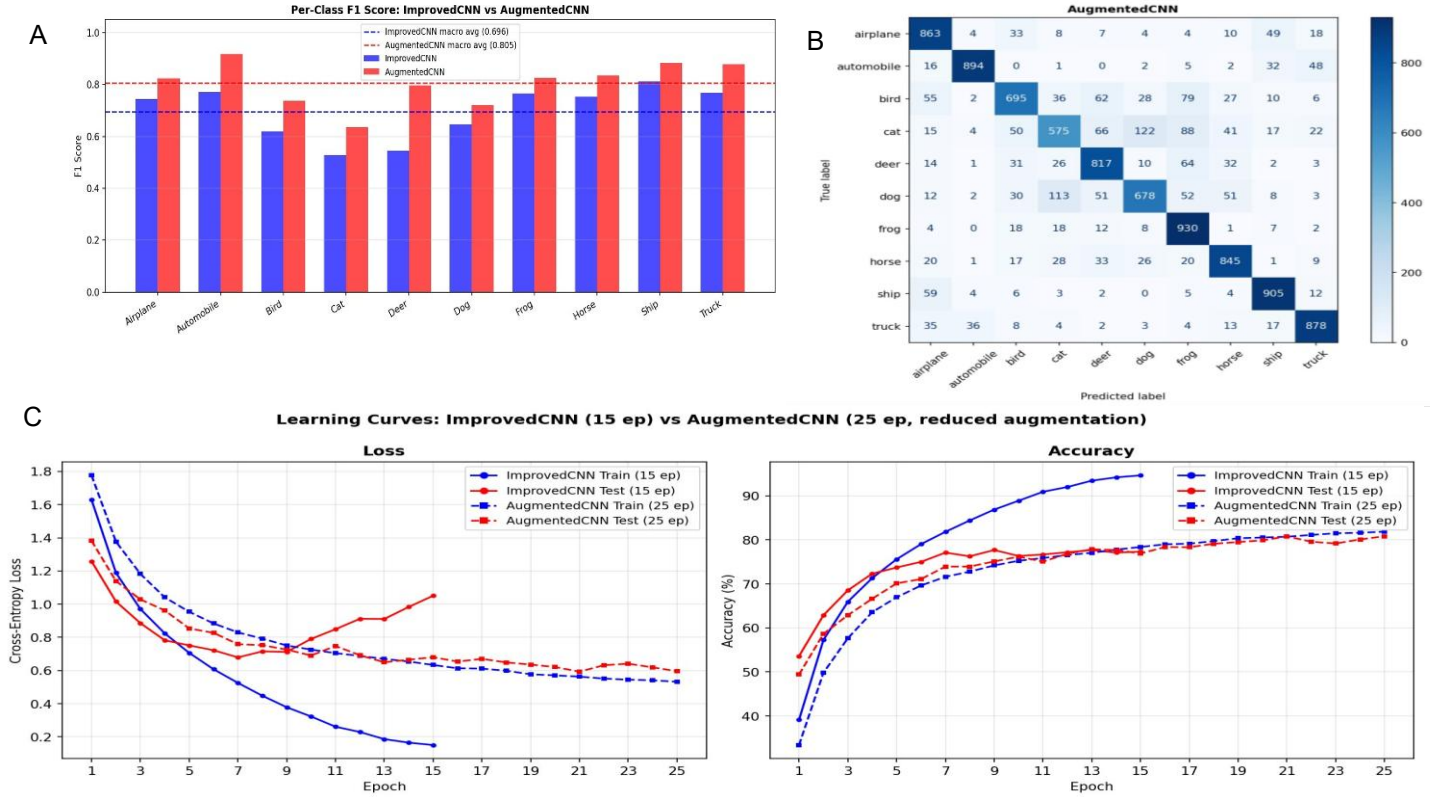


Figure 6. (A) Per-class F1 score for ImprovedCNN (blue bars) and AugmentedCNN (red bars). Dashed lines show macro F1 averages for each model. (B) Confusion matrices for AugmentedCNN. Values show raw counts out of 1,000 test samples per class. (C) Loss and accuracy learning curves for ImprovedCNN (solid lines, 15 epochs) and AugmentedCNN (dashed lines, 25 epochs). Blue = train, red = test.

3. Learning Curves

The two models exhibit opposite overfitting patterns (Figure 6(C)). ImprovedCNN's train accuracy diverges sharply from test reaching a 19.81% gap, the classic overfitting signature discussed in section 3. AugmentedCNN's train and test curves rise together, maintaining a near-zero gap of 1.05%, with both loss curves decreasing consistently to 0.5953 test loss at epoch 25, well below ImprovedCNN's final test loss of 1.0925, reflecting better calibrated predictions on unseen images, not just higher accuracy. Training was extended to 25 epochs because augmentation produces a noisier learning signal, requiring more repetitions to converge. The curves confirm this, improving through epoch 20 before plateauing, beyond which further training yields diminishing returns.

5. Discussion

Augmentation with 25 epochs produces a clear improvement over the ImprovedCNN baseline on every metric. The three selected augmentations, random crop, zoom, and horizontal flip, were chosen for their gentleness. At 32×32 resolution, aggressive augmentation destroys meaningful content, so positional and orientation variety without signal corruption proved the correct trade-off. The 5.60% point gain in test accuracy and the collapse of the train/test gap to 1.05% confirm that augmentation does not merely reduce overfitting, it improves generalization by forcing the model to learn position-invariant features rather than memorize fixed pixel patterns, as if trained on a much larger dataset. Macro F1 and AUC-ROC confirm the improvement is consistent across all classes. Compared to early stopping and weight decay, augmentation is complementary. Those methods constrain the optimiser, while augmentation expands the training distribution. Combining all three would likely push test accuracy further.

5. Creativity Task - PatchMLP_CNN

1. Motivation and Architecture

Throughout our report, visually similar class pairs were consistently the hardest to classify correctly. These classes share the same broad shape and colouration, and what separates them are localised differences such as ear shape, facial texture, or the shape of a wheel arch. A standard CNN scans the entire image with the same filter at every position, powerful for global features but blind to localised differences. A standard MLP flattens the image completely and loses all spatial structure before processing begins. The PatchMLP_CNN (Figure 7) is designed to address this directly. The central idea is to divide the image into spatial regions and learn about each region independently, then combine with the global feature extraction of the CNN. To isolate the contribution of the patch-based MLP branch, the CNN branch is kept identical to ImprovedCNN in every respect: same architecture, same hyperparameters, same data preprocessing, and same training setup. No additional preprocessing steps such as a separate validation split or further augmentation were introduced. The only structural addition is the MLP branch and the fusion layer that combines the two embeddings. This ensures that any difference in performance between PatchMLP_CNN and ImprovedCNN can be attributed directly to the patch decomposition design rather than to other changes.

2. MLP Branch

Each 32×32 RGB image is divided into four non-overlapping 16×16 patches. Each patch contains $3 \times 16 \times 16 = 768$ pixel values, which are flattened into a vector and processed by its own MLP. The four MLPs do not share weights because each one specialises in the patterns typical of its own quadrant, unlike the CNN which applies the same filter across the entire image. Each patch MLP consists of two fully connected layers. Following the findings of the CNN Debugging section, ReLU is used as the activation function throughout, to avoid weakened gradients by maintaining a gradient of 1 for all positive inputs. Batch Normalisation is applied after each linear layer to keep activations at a stable scale, preventing early convergence plateau. A Dropout layer (rate 0.4) after the first fully connected layer provides regularisation. A rate of 0.4 was chosen rather than the 0.5 used in the CNN branch, as a higher rate risked losing information in the smaller per-patch inputs. The outputs of the four patch MLPs are combined to form a 512-dimensional embedding that encodes where in the image particular patterns appear.

3. CNN Branch, Fusion and training setup

The CNN branch is identical to the ImprovedCNN used throughout the report: four convolutional layers (each followed by ReLU and 2×2 max-pooling), a fully connected layer with ReLU, and Dropout (rate 0.5) with the final classification layer removed, its 256-dimensional output passed instead to the fusion head. After four pooling operations the spatial resolution has been reduced from 32×32 to 2×2 , so the CNN branch produces a 256-dimensional global embedding that summarises what the most prominent features in the image are, regardless of location.

The 512-dimensional MLP embedding and the 256-dimensional CNN embedding are combined into a single 768-dimensional vector. This fused vector is passed through two fully connected layers with Batch Normalisation, ReLU, and Dropout (rate 0.4), producing the final class probabilities.

The model is trained for 15 epochs using the Adam optimiser at a learning rate of 0.001. Weight decay ($\lambda = 1 \times 10^{-4}$) is added as a regularisation measure. This modification was not applied in ImprovedCNN, where Dropout alone was enough. Here, the larger parameter count motivates an extra safeguard against overfitting, at a low enough value not to interfere with learning. The same normalised CIFAR-10 data loaders used in all previous sections are reused here (mean and standard deviation computed from the training set), ensuring a fair comparison.

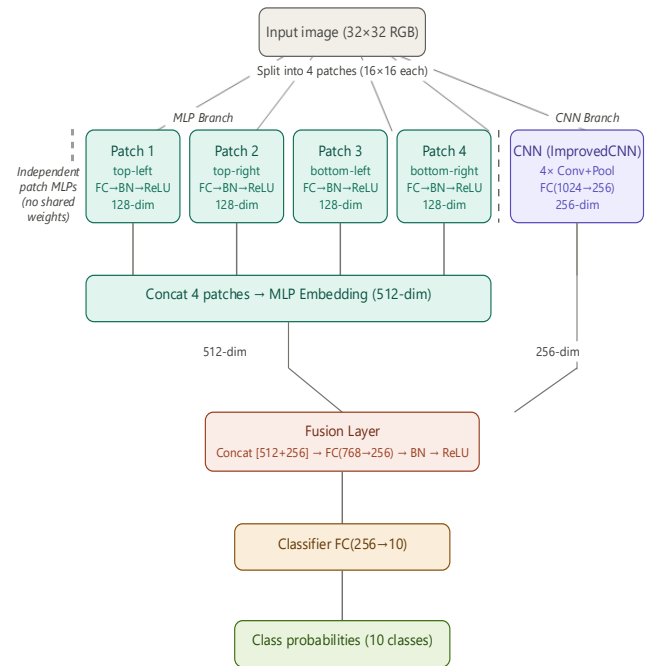


Figure 7. Architecture of the PatchMLP_CNN hybrid network.

4. Evaluation and Analysis

PatchMLP_CNN achieves a higher test accuracy (77.31% vs. 75.20%) while simultaneously reducing the train/test gap from 19.81% to 14.80%. The small drop in training accuracy (95.01% to 92.11%) reflects that the model is less prone to memorisation.

5. Learning Curves

Both models follow the same general pattern: rapid improvement in early epochs, then a gradual plateau as the model converges (Figure 8(A)). The gap between train vs. test for PatchMLP_CNN is visibly narrower than the equivalent gap for ImprovedCNN, confirming the reduction in overfitting. PatchMLP_CNN's test loss is lower and more stable in the later epochs, further evidence of reduced overfitting. This stability is partly caused by the weight decay regularisation added to PatchMLP_CNN's optimiser.

6. Per-Class F1 Scores

The per-class F1 chart and numerical comparison (Figure 8(B)) directly address our goal for the new model. Every single class improves. The gains are largest for the classes that were hard throughout the project: deer (+0.191), truck (+0.179), and bird (+0.116). Deer was the weakest ImprovedCNN class (F1 = 0.523), and the patch design provides the greatest benefit there (as with augmentation). Cat remains the single weakest class overall (F1 = 0.595), but even this improves by 0.066 compared to ImprovedCNN. The cat/dog confusion that appeared in every confusion matrix throughout the report is partially reduced. Examining the PatchMLP_CNN confusion matrix (*see Figure A6, in appendix*), the dog row shows 658 correct predictions compared to 588 for ImprovedCNN, a substantial 70-prediction gain.

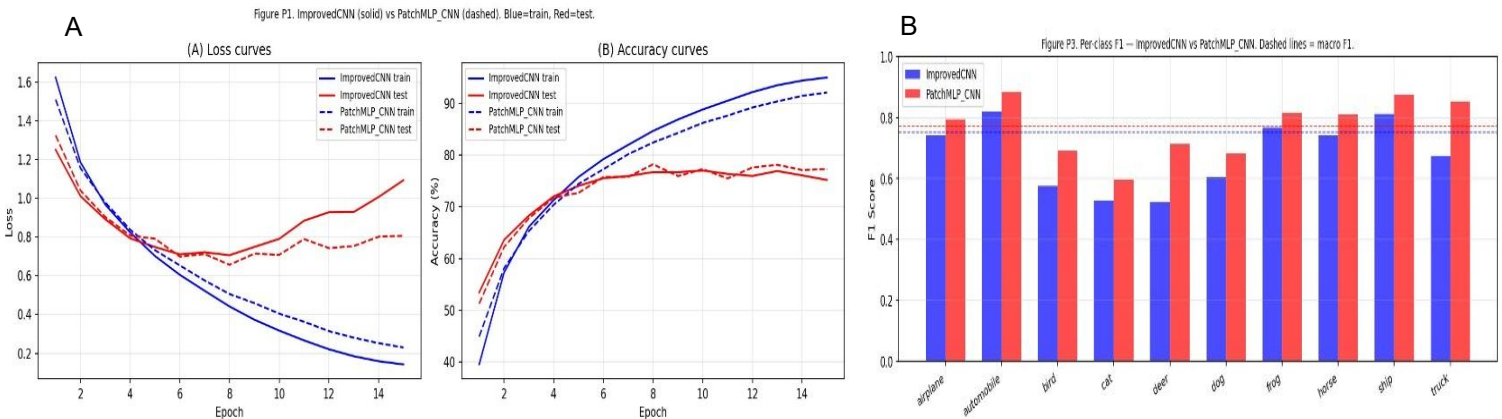


Figure 8. (A) Loss and accuracy learning curves of ImprovedCNN and PatchMLP_CNN. (B) Per-class F1 - ImprovedCNN (blue) vs PatchMLP_CNN (red).

7. Discussion

By combining a CNN's ability to extract global features with an MLP's ability to specialise in individual spatial regions, the model improves on every class, reduces the train/test gap, and shows particular gains on the subtle differences between visually similar categories that were the main weakness of the baseline models. The largest improvements align with the design intuition: deer and horses are distinguished largely by leg structure and body proportions distributed across spatial quadrants, and trucks and automobiles differ most clearly in localised regions such as the upper body and wheel arch.

Three design choices contributed to the improved generalisation relative to ImprovedCNN: (1) the patch decomposition forces the network to develop region-specific representations, increasing the diversity of learned features. (2) Batch Normalisation in the MLP and fusion layers stabilises gradients and allows each component to train at a consistent scale. and (3) weight decay in the optimiser provides continuous regularisation, reducing the tendency to memorise training-specific patterns.

There are also natural limitations. The 16×16 patch size does not adapt to the content of the image. If an object of interest falls across the boundary between two patches, no single MLP sees it whole. A more flexible approach might use overlapping patches or allow the network itself to decide which regions of the image deserve more attention — rather than dividing it into fixed equal quadrants regardless of content. Additionally, the model is somewhat larger than ImprovedCNN (821K vs. 653K parameters), though both are far smaller than the InitialMLP (1.7M parameters) and both achieve better accuracy. Future work could explore whether further regularisation (larger dropout, data augmentation) or a deeper fusion network could close the remaining gap between cat and dog, which continues to be confused despite the improvements shown here.

APPENDIX

1. Initial CNN

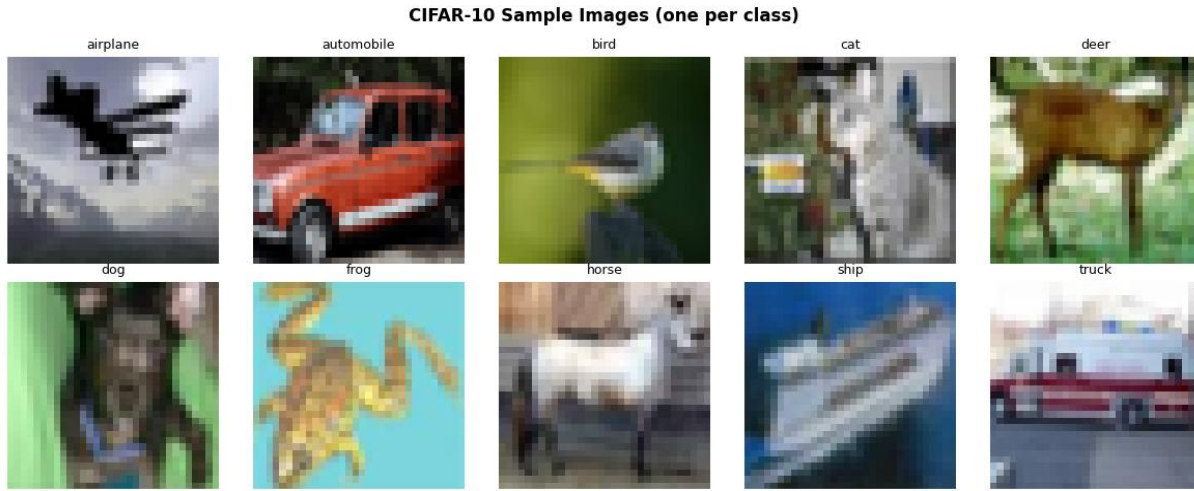


Figure A1: Sample images from CIFAR-10 (one per class).

Epoch	Train Loss	Train Acc	Test Loss	Test Acc
1	2.1637	17.66%	1.9575	26.68%
2	1.8826	30.63%	1.7411	36.57%
3	1.6843	38.61%	1.5862	42.52%
4	1.5383	44.10%	1.5129	44.89%
5	1.4446	47.56%	1.4012	49.31%
6	1.3658	50.42%	1.3312	52.14%
7	1.3125	52.36%	1.2780	53.16%
8	1.2534	54.74%	1.2437	54.82%
9	1.2005	56.55%	1.2094	56.27%
10	1.1512	58.58%	1.1562	58.14%
11	1.0965	60.61%	1.1312	59.06%
12	1.0545	62.29%	1.0944	60.84%
13	1.0078	63.68%	1.0852	61.45%
14	0.9625	65.67%	1.0642	62.13%
15	0.9161	67.43%	1.0381	63.17%

Final Train Accuracy : 67.43%
Final Test Accuracy : 63.17%
Final Train Loss : 0.9161

Table A1: Epoch-by-epoch training and test results for InitialCNN over 15 epochs.

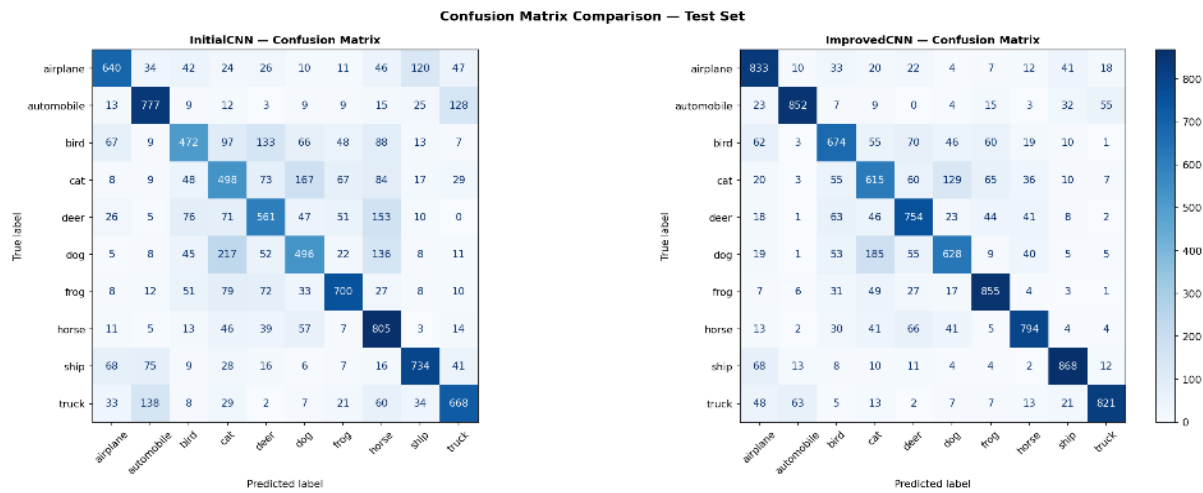


Figure A2: Confusion matrices for InitialCNN and ImprovedCNN on the CIFAR-10 test set.

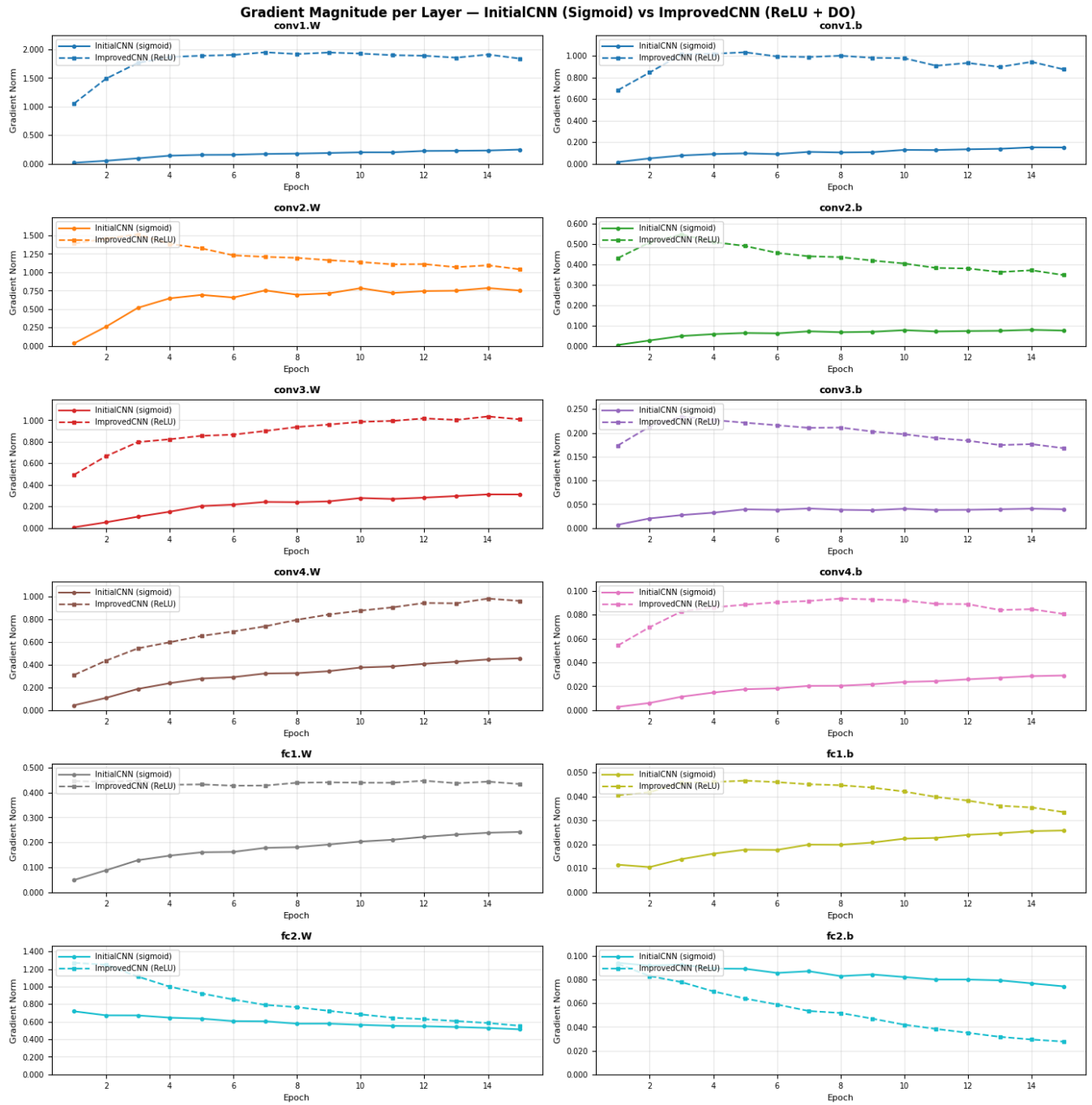


Figure A3. Gradient magnitude per layer across 15 epochs — InitialCNN with Sigmoid vs ImprovedCNN with ReLU and Dropout.

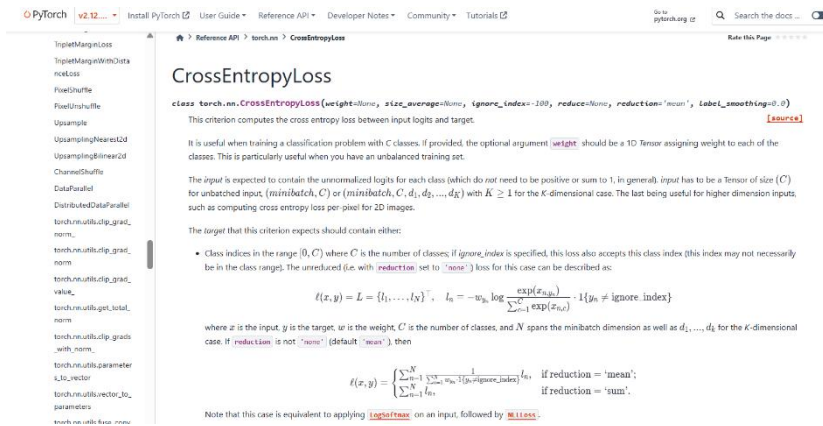


Figure A4. Official PyTorch documentation for CrossEntropyLoss, indicating that the loss function internally applies LogSoftmax before NLLoss.

Epoch	Train Loss	Train Acc	Test Loss	Test Acc
1	1.6225	39.55%	1.2502	53.45%
2	1.1855	57.28%	1.0107	63.60%
3	0.9667	66.14%	0.8909	68.32%
4	0.8235	71.32%	0.7930	72.04%
5	0.7018	75.78%	0.7482	74.06%
6	0.6048	79.24%	0.7103	75.54%
7	0.5222	82.00%	0.7209	75.96%
8	0.4418	84.68%	0.7061	76.73%
9	0.3733	86.88%	0.7487	76.71%
10	0.3169	88.83%	0.7902	77.03%
11	0.2663	90.52%	0.8836	76.37%
12	0.2208	92.20%	0.9276	75.98%
13	0.1849	93.51%	0.9289	76.93%
14	0.1591	94.41%	1.0062	76.09%
15	0.1426	95.01%	1.0925	75.20%

Final Train Accuracy : 95.01%
Final Test Accuracy : 75.20%

Table A2: Epoch-by-epoch training and test results for ImprovedCNN over 15 epochs.

2. MLP

...	Epoch	Train Loss	Train Acc	Test Loss	Test Acc	Time(s)
	1	1.6287	42.20%	1.4698	48.15%	32.8s
	2	1.4109	50.23%	1.4259	49.30%	32.6s
	3	1.2981	54.26%	1.3630	52.16%	32.2s
	4	1.2050	57.53%	1.3686	52.78%	33.2s
	5	1.1172	60.52%	1.3475	53.15%	32.2s
	6	1.0368	63.35%	1.3478	54.13%	33.3s
	7	0.9578	65.93%	1.3772	54.19%	32.9s
	8	0.8837	68.70%	1.4284	54.11%	32.2s
	9	0.8088	71.26%	1.4953	53.66%	33.3s
	10	0.7292	74.24%	1.5275	53.68%	33.2s
	11	0.6623	76.63%	1.5868	53.97%	32.4s
	12	0.6044	78.61%	1.7483	52.48%	33.3s
	13	0.5409	80.80%	1.7478	53.86%	33.0s
	14	0.4870	82.81%	1.8643	52.85%	33.1s
	15	0.4407	84.42%	1.9829	53.18%	33.2s

Final Train Accuracy : 84.42%
Final Test Accuracy : 53.18%
Final Train Loss : 0.4407
Avg time per epoch : 32.9s
Peak memory (MLP) : 2.1 MB

Table A3: Epoch-by-epoch training and test results for MLP over 15 epochs.

MLP – F1 Score per class:

airplane	0.574
automobile	0.642
bird	0.439
cat	0.376
deer	0.469
dog	0.450
frog	0.603
horse	0.588
ship	0.656
truck	0.576

Macro F1: 0.537

Macro AUC-ROC: 0.890

Comparison – ImprovedCNN vs MLP:

Class	CNN F1	MLP F1	Change
airplane	0.790	0.574	-0.215
automobile	0.865	0.642	-0.223
bird	0.657	0.439	-0.218
cat	0.567	0.376	-0.191
deer	0.710	0.469	-0.242
dog	0.654	0.450	-0.204
frog	0.810	0.603	-0.207
horse	0.792	0.588	-0.204
ship	0.848	0.656	-0.192
truck	0.840	0.576	-0.264
Macro F1	0.753	0.537	-0.216

Table A4. Per-class F1 scores for the MLP model, macro F1, macro AUC-ROC, and a direct comparison against ImprovedCNN.

4. Augmentation:



Figure A5. Effect of each augmentation on a sample CIFAR-10 image (horse). Each row shows one of the three applied transformations with five different random seeds. Column 1 is the unmodified original.

Epoch	Train Loss	Train Acc	Test Loss	Test Acc
1	1.7762	33.41%	1.3817	49.42%
2	1.3768	49.81%	1.1388	58.62%
3	1.1823	57.65%	1.0285	62.89%
4	1.0409	63.57%	0.9621	66.62%
5	0.9538	66.94%	0.8534	70.11%
6	0.8835	69.62%	0.8268	71.08%
7	0.8293	71.60%	0.7583	73.92%
8	0.7919	72.78%	0.7522	73.93%
9	0.7506	74.18%	0.7257	75.09%
10	0.7257	75.25%	0.6903	76.17%
11	0.7044	75.86%	0.7476	75.06%
12	0.6877	76.54%	0.6917	76.82%
13	0.6696	77.05%	0.6493	77.86%
14	0.6540	77.76%	0.6647	77.67%
15	0.6331	78.37%	0.6802	76.89%
16	0.6132	78.98%	0.6534	78.35%
17	0.6112	79.13%	0.6700	78.32%
18	0.5981	79.71%	0.6484	79.10%
19	0.5769	80.36%	0.6355	79.49%
20	0.5705	80.52%	0.6207	79.87%
21	0.5627	80.70%	0.5927	80.78%
22	0.5509	81.11%	0.6317	79.56%
23	0.5446	81.48%	0.6407	79.16%
24	0.5405	81.60%	0.6186	80.09%
25	0.5319	81.85%	0.5953	80.80%

Final Train Accuracy : 81.85%
 Final Test Accuracy : 80.80%
 Train/Test Gap : 1.05%

Table A5: Epoch-by-epoch training and test results for AugmentedCNN over 25 epochs.

Metric	ImprovedCNN	AugmentedCNN
Final Train Accuracy	94.62%	81.85%
Final Test Accuracy	77.32%	80.80%
Train/Test Gap	17.30%	1.05%
Final Train Loss	0.1504	0.5319
Final Test Loss	1.0504	0.5953
Macro F1	0.696	0.805
Macro AUC-ROC	0.951	0.979

Per-class F1:	ImprovedCNN	AugmentedCNN	Δ
Airplane	0.745	0.825	+ 0.080
Automobile	0.772	0.918	+ 0.146
Bird	0.619	0.736	+ 0.117
Cat	0.529	0.635	+ 0.105
Deer	0.545	0.796	+ 0.251
Dog	0.645	0.721	+ 0.075
Frog	0.766	0.826	+ 0.060
Horse	0.754	0.834	+ 0.080
Ship	0.813	0.884	+ 0.071
Truck	0.768	0.878	+ 0.109

Table A6: ImprovedCNN vs AugmentedCNN final metrics and per-class F1(epoch 25)..

5. Creativity Task

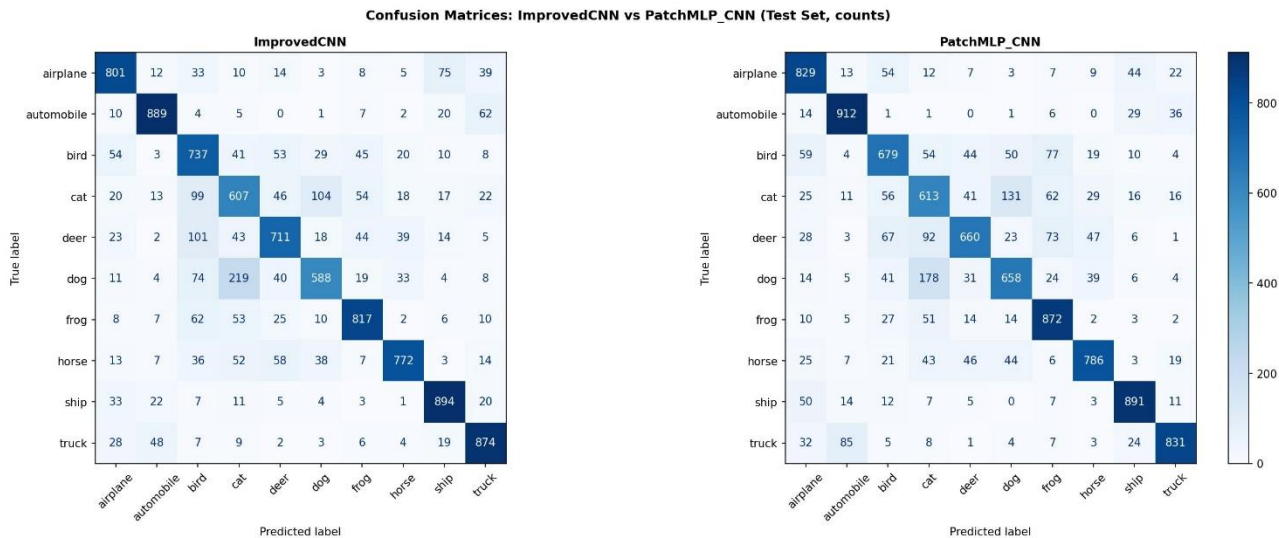


Figure A6. Confusion matrices — ImprovedCNN (left) vs PatchMLP_CNN (right).

Epoch	Tr Loss	Tr Acc	Te Loss	Te Acc
1	1.5090	44.95%	1.3243	51.30%
2	1.1550	58.14%	1.0400	62.23%
3	0.9784	65.31%	0.9031	67.78%
4	0.8378	70.42%	0.8076	71.79%
5	0.7318	74.45%	0.7910	72.72%
6	0.6534	77.28%	0.6989	75.77%
7	0.5754	80.19%	0.7120	75.80%
8	0.5056	82.44%	0.6563	78.23%
9	0.4590	84.29%	0.7140	75.96%
10	0.4045	86.23%	0.7079	77.29%
11	0.3628	87.64%	0.7890	75.50%
12	0.3140	89.22%	0.7426	77.62%
13	0.2807	90.39%	0.7530	78.14%
14	0.2519	91.47%	0.8012	77.11%
15	0.2301	92.11%	0.8060	77.31%

Training complete.

Table A7: Epoch-by-epoch training and test results for PatchMLP_CNN over 15 epochs.

Metric	ImprovedCNN	PatchMLP_CNN
Final Train Accuracy	95.01%	92.11%
Final Test Accuracy	75.20%	77.31%
Train/Test Gap	19.81%	14.80%
Macro F1	N/A	0.772
Macro AUC-ROC	N/A	0.971

Class	ImprovedCNN	PatchMLP_CNN	Delta
airplane	0.743	0.795	+ 0.052
automobile	0.819	0.886	+ 0.067
bird	0.576	0.692	+ 0.116
cat	0.529	0.595	+ 0.066
deer	0.523	0.714	+ 0.191
dog	0.606	0.683	+ 0.077
frog	0.768	0.815	+ 0.047
horse	0.742	0.812	+ 0.070
ship	0.812	0.877	+ 0.065
truck	0.675	0.854	+ 0.179

Table A8: ImprovedCNN vs PatchMLP_CNN final metrics and per-class F1 (epoch 15).